Kathmandu University

Department of Computer Science and Engineering

Dhulikhel, Kavre

A Project Report

on

"Expression Converter"

[Code No: COMP 202]

(For partial fulfillment of __II__ Year/__I__in Computer Engineering)

Submitted by

Pragyee Dahal (Roll No. 10)

Submitted to

Er. Sagar Acharya

Department of Computer Science and Engineering

Submission Date: 2026-02-24

Acknowledgement

I would like to express my sincere gratitude to **Er. Sagar Acharya**, Lecturer at the Department of Computer Science and Engineering, Kathmandu University, for his consistent guidance and support throughout this mini-project. His insights were instrumental in shaping the implementation of the expression conversion logic.

I am also thankful to the Department of Computer Science and Engineering for providing a conducive learning environment and the necessary resources. Finally, I would like to thank my friends and family for their constant motivation and encouragement during the development of this project.

Abstract

This project, Expression Converter is a desktop application that bidirectionally converts mathematical expressions between Infix, Postfix, and Prefix notations. It was created with C++ and the Qt framework. Two basic data structures are implemented from scratch by the system: a singly linked list for storing conversion history and a stack utilized in the Shunting-Yard technique for expression conversion. The application's real-time step-by-step algorithm visualization, which shows each push and pop action in a tabular style following each conversion, is one of its primary features. The application runs entirely offline and features a professional dark-themed graphical user interface with color-coded controls. For computer science students learning about data structures, algorithms, and compiler design, Expression Converter is a useful teaching tool.

Keywords: *Data Structures and Algorithms, Stack (LIFO), Linked List, Shunting-Yard Algorithm, Infix Notation, Postfix Notation, Prefix Notation, Qt Framework, Algorithm Visualization*

Table of Contents

Abstract ii

List of figures iv

List of Abbreviations v

Chapter 1 Introduction 1

1.1 Background 1

1.2 Objectives 1

1.3 Motivation and Significance 2

Chapter 2 Related Works 3

Chapter 3 Design and Implementation 5

3.1 System Architecture 5

3.2 Data Structures 6

3.3 Algorithm Implementation 7

3.4 Use Case Diagram 9

Chapter 4 System Requirement Specifications 10

4.1 Software Specifications 10

4.2 Hardware Specifications 10

Chapter 5 Discussion on Achievements 12

5.1 Features 12

Chapter 6 Conclusion and Future Works 14

6.1 Limitations 14

6.2 Future Enhancements 15

References 17

Appendix 18

List of figures

Figure 1 Use Case Diagram 9
Figure 2 Main Window 18
Figure 3 Infix to Postfix 19
Figure 4 Infix to Prefix 20
Figure 5 Postfix to Infix 21
Figure 6 Prefix to Infix 22

List of Abbreviations

DSA	Data Structures and Algorithms
IDE	Integrated Development Environment
LIFO	Last in First Out
RAM	Random Access Memory
SSD	Solid State Drive
UI	User Interface

Chapter 1 Introduction

1.1 Background

An expression is any word or group of words or symbols that generates a value on evaluation (Tutorialspoint, n.d.). Expressions can be written in three notations: Infix; where operators appear between operands, Postfix where operators follow operands, Prefix where operators precede operands. Postfix and prefix are much simpler for machines to evaluate without extra parsing rules, whereas infix is intuitive for humans.

Expression Converter is a Qt-based desktop application built using C++ which enables conversion from infix to postfix and prefix and vice versa. This project implements conversion between notations using concepts of DSA. The standard algorithm used for the conversion is Shunting Yard algorithm, developed by computer scientist, programmer Edsger Wybe Dijkstra. The project employs two data structure implementations: Stack used in Shunting Yard Algorithm for conversion and Singly Linked List for storing conversion history.

1.2 Objectives

The main objective of developing the project Expression Converter are:

1. To demonstrate practical applications of Stack data structure.
2. To apply Shunting Yard Algorithm for computing expression conversions.
3. To implement Singly Linked List for storage and management of conversion history.
4. To provide real time algorithm visualization with step-by-step execution of conversions.

1.3 Motivation and Significance

The idea of developing Expression Converter arose from a common challenge faced by students to understand Shunting Yard algorithm while conversion of expressions. Students faced challenges in understanding how different data structures like Stack behave step by step during actual execution. The existing online converters provide results directly without showing the steps during conversion. Most available converters are web based and requires an active internet connection, which limits their accessibility.

This project is significant because it addresses all these shortcomings in one application. It provides visual feedback for understanding complex algorithms. It demonstrates real world application of DSA concepts. By displaying a complete step by step trace of every push, pop and output during conversion, it allows students to understand its algorithm. Application of Singly Linked List helps in having conversion history making learning experience easy for students. It shows how classical algorithm can be integrated with UI frameworks for creating practical applications. Unlike online converters, the application runs entirely offline, making it accessible at any time.

Chapter 2 Related Works

The conversion of expressions has been a long-researched subject in computer science, and there are a number of tools and implementations that do this job in one way or another. This chapter examines the existing solutions that seem to be the most applicable such as an online converter, console-based implementations, as well as educational software to define their capabilities and limitations.

The most convenient converters are web based online converters which enable users to convert expressions easily using a browser interface. Most of these tools normally need an active internet connection, only one direction of conversion and the final result is displayed without showing the process that the algorithm took in arriving at the result.

CalConT is a web based online expression converter that performs conversion of all three notations. It offers a simple interface where users can enter the expressions and obtain the equivalent output. But it does not provide a step-by-step visualization of the process of conversion and there is no record of past conversions as well.

Expression Tree Generator is a web based online tool that generates tree from mathematical expressions. It lacks the essential capability of conversion of all three notations, despite the fact that it can be helpful for creating test inputs. There is no stack trace, no history management of any type, and no visualization of the method.

Free Online Calculator is an online expression converter which along with conversion also provides a brief textual explanation of the algorithm. Instead of a dynamic, step-by-step depiction connected to the actual input being transformed, the explanation is provided as static text. Real-time stack operations and the way the operator stack varies at each token during conversion are not shown by the tool.

Scanftree is also an available online converter offering multiple conversions which shows only simple conversion without a detailed visualization. Instead of an interactive, tabular visualization, the stages are displayed as plain text output. The interface does not make clear how each token being processed relates to the push, pop,

and top stack operations. Moreover, the absence of a history management feature prevents users from reviewing or comparing previous conversions.

The Expression Converter developed addresses the limitations identified in these tools. With the help of its real-time algorithm visualization and comprehensive, tabular, step-by-step trace, users can see precisely how the stack changes at each stage of the conversion process. The custom Singly Linked List implementation is used to maintain the conversion history, the data structure is no longer merely a background mechanism but rather an integral part of the learning process. The codebase exposes clear DSA implementations that students can study alongside the application. With its dark theme and color-coded controls, the interface is both aesthetically pleasing for prolonged usage and simple to operate. When combined, these features make the Expression Converter more than just a conversion tool; it's a comprehensive learning resource for students learning algorithms and data structures.

Chapter 3 Design and Implementation

This chapter discusses the system architecture, data structure implementations, and algorithm design of the Expression Converter. Each component was designed with clarity and educational value in mind to guarantee that the implementation not only works as intended but also demonstrates how fundamental DSA concepts are applied in practical situations.

3.1 System Architecture

The system follows a modular architecture made up of five different layers. The application design was chosen on purpose so that each part of the application can be developed, tested, modified independently without affecting the other parts. For example, if we make changes to the algorithm layer, we do not have to make any changes to the user interface. This separation ensures that updates to the Shunting-Yard logic do not require changes to the Qt-based presentation layer.

3.1.1 Data Structure Layer

Expression converter implements custom Singly Linked List and Stack with all standard operations.

3.1.2 Algorithm Layer

The conversion in the system is built on Shunting Yard Algorithm and stack-based conversions.

3.1.3 Business Logic Layer

The system handles the conversion dispatching along with history management.

3.1.4 Presentation Layer

Expression Converter is built using Qt and has single slot mechanism for user interaction and event handling.

3.1.5 Visualization Layer

It generates real time step-by-step algorithm traces in a custom-made table.

3.2 Data Structures

To demonstrate a deep understanding of memory management, a custom Stack class was implemented from scratch using a Singly Linked List. This ensures $O(1)$ time complexity for push and pop operations without relying on the C++ Standard Template Library.

3.2.1 Stack

Stack is fundamental data structure used in the project to explain the concept of DSA along with practical visualization. Stack is the data structure that uses the concept of LIFO. It is implemented as a template class using QVector as the storage mechanism. Stack, in this project has time complexity of O(1) for push and pop operations.

Operations implemented on Stack are:

1. **push(T val):** Adds an element to top
2. **pop():** Removes the top element if stack is not empty
3. **top():** Returns the top element without removing it
4. **isEmpty():** Checks if the stack is empty

Stack is used for:

1. **Infix to Postfix**
   It holds the operators that are waiting to be sent to the output and are arranged according to their precedence level.
2. **Infix to Prefix**
   Stack is used here inside Shunting Yard algorithm after the input expression is reversed and its parentheses are swapped to ensure that the reversal trick produces accurate output.

3. **Postfix/Prefix to Infix:**
   Stack now holds partial QString subexpressions to complete the conversion of infix expression.

3.2.2 Singly Linked List

Singly Linked List is a type of Linked List data structure which is used in this project to maintain a dynamic history of all the conversions performed in a session. In Singly Linked List the node stores the address of the next node. Here, in this project each node is represented using HistoryNode struct.

The operations implemented on Linked List are:

1. **insertFront():** It creates a new node and ensures the most recent conversion always appears first in the history panel. Time complexity: O(1)
   insertFront() was chosen over insertBack() so that the history panel has the most recent conversion at the top for a user friendly experience.
2. **getAll():** Traverses the full list from head to tail and returns all nodes in a QVector for display. Time complexity: O(n)

3.3 Algorithm Implementation

The conversion system implements four algorithms for bidirectional conversion between the notations.

3.3.1 Shunting Yard Algorithm (Infix to Postfix)

Shunting Yard algorithm is the core algorithm concept of Expression Converter. Edsger Dijkstra developed his "Shunting Yard" algorithm to convert an infix expression into a postfix expression (Wolf, n.d.). The algorithm processes the infix notation from left to right applying following rules:

- Operands (letters or numbers) are added directly to the output.
- Left parenthesis '(' is pushed onto the stack.

- Right parenthesis ')' triggers popping of operators until left parenthesis '(' is found.
- Operators are handled based on precedence and associativity.
- All remaining operators on the stack are popped to the output.

3.3.2 Infix to Prefix

This process reuses the Shunting Yard Algorithm by simply reversing it and applying the algorithm. The algorithm processes the conversions applying following rules:

- Reverse the infix expression character by character.
- Swap parentheses: every '(' becomes ')' and every ')' becomes '('.
- Apply the Shunting-Yard algorithm to the modified string to obtain an intermediate postfix result.
- Reverse the resulting token list to obtain the final prefix expression.

3.3.3 Postfix to Infix

For the conversion of postfix notation to infix notation the system uses Stack (QString) to build a sub expression and reverse the evaluation process and computes it. The algorithm processes the notation by scanning from left to right and the conversions applying following rules:

- Operands are pushed directly onto the string stack.
- When an operator is encountered: two operands are popped (b first, then a), combined as the string "(a operator b)", and the result is pushed back onto the stack.
- After all tokens are processed, the single remaining element on the stack is the complete infix expression with explicit parentheses.

3.3.4 Prefix to Infix

The conversion from prefix to infix is similar to conversion of postfix to infix but the expression is processes from right to left. The algorithm processes the conversions applying following rules:

- Operands are pushed onto the string stack.
- When an operator is encountered: two operands are popped (a first, then b), combined as "(a operator b)", and pushed back.
- The single remaining element on the stack is the reconstructed infix expression.

3.4 Use Case Diagram

The figure below represents the use case diagram of Expression Converter illustrating the interaction of actor (User) with the system. The user first enters the expression and the user can trigger four conversion cases: Infix to Postfix, Infix to Prefix, Postfix to Infix, Prefix to Infix. The conversion can be understood by the algorithm visualization present in the system. Each conversion is stored in the history panel which can also be cleared.

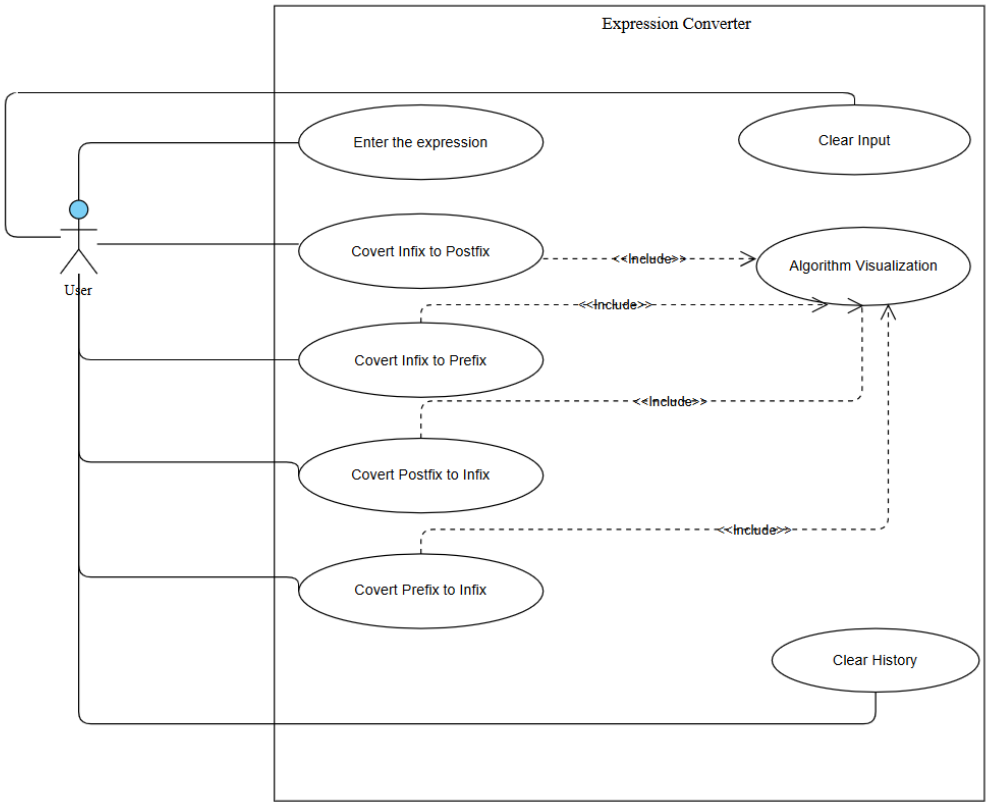

Figure 1 Use Case Diagram

Chapter 4 System Requirement Specifications

This chapter discusses the requirements necessary for the successful deployment of project. System requirements are divided into Software Specifications and Hardware Specifications to ensure that the application runs correctly on the target machine.

4.1 Software Specifications

The following software components are required to run and build the Expression Converter:

- Operating System: Windows 10, Ubuntu 20.04+, or macOS 10.15+
- Programming Language: C++
- Framework: Qt Framework Version 6.0
- IDE: Qt Creator Version 6.0
- Build System: qmake

4.2 Hardware Specifications

The following component outlines the hardware configurations for running the application:

Minimum Configuration:

- Processor: Intel Core i3 or equivalent
- RAM: 4 GB
- Storage: 200 MB free disk space
- Display: 1366×768 resolution
- Operating System: Windows 10, Ubuntu 20.04+, or macOS 10.15+

Recommended Configuration:

- Processor: Intel Core i5 or higher

- RAM: 8 GB
- Storage: SSD with at least 1 GB free space
- Display: 1920×1080 resolution
- Operating System: Windows 11

Chapter 5 Discussion on Achievements

5.1 Features

Expression Converter is a desktop-based application which is developed to offer a comprehensive set of features for converting different notations like Infix, Postfix, and Prefix. In order to help students grasp fundamental DSA principles, the system combines technical and instructional elements, merging real-time algorithm display with custom data structure implementations.

5.1.1 Bidirectional Expression Conversion

The main feature of the system is that the user can enter an expression and choose to convert it from Infix to Postfix, Infix to Prefix and also convert the expression from Postfix to Infix, Prefix to Infix, in this way the conversion is bidirectional. Each conversion correctly handles the operator precedence and associativity. This feature helps the user by never needing to switch between multiple tools for different conversions.

5.1.2 Algorithm Visualization

Another main feature of Expression Converter is the real time algorithm visualization. After every conversion, the algorithm visualizer panel displays step by step conversion process along with a table for that particular conversion. The table shows exactly how algorithm is executed including the number of steps, current token, stack and output. For conversions of Postfix to Infix and Prefix to Infix, the table shows the operations along with descriptions of push and pop.

5.1.3 History Management

Expression Converter maintains a complete history of all the conversions executed during a session using the custom Singly Linked List. The new conversions are inserted at the front of the list displaying it on the top in history panel. The history panel also

displays the number of entries, current linked list node counts. The user can also clear the history themselves.

Chapter 6 Conclusion and Future Works

Expression Converter successfully demonstrates how the concept of DSA can be applied in a practical visual application. The project implements the bidirectional conversion of the different notations along with a real time algorithm visualization. The system implements the concept of Stack for the conversions and Singly Linked List for the conversion history. Shunting Yard Algorithm is used for the optimal conversions. The system offers a thorough learning environment where students may see how algorithms work step-by-step in addition to converting expressions.

Expression Converter is designed using Qt Creator and a dark navy-blue theme is maintained across all panels. The background has dark theme in order to reduce eye strain during extended use. The conversion buttons are color coded into blue and green so the users can identify actions at a glance.

6.1 Limitations

Despite the successful deployment of the project, it has certain limitations

6.1.1 Single Character Operands

The current implementation of project supports single character operands only like A, B, C. Multi digit numbers or variables like abc or x1, y1 are not supported. This limitation restricts the practical use of converter in educational purposes.

6.1.2 No History Persistence

The conversion history is stored for an active session only. Once the user exits the Qt output, the history is cleared as there are is no file base database storage implementation. Users cannot retrieve the conversions from previous sessions.

6.1.3 Limited Operator Support

Expression converter currently only supports basic arithmetic operations like addition, subtraction, multiplication, division and exponentiation. The system does not support logical operators like AND and OR.

6.1.4 No Undo/Redo Functionality

There is no undo or redo option for conversions in the application. The user cannot return to a prior state after a conversion has been completed without manually re-entering the expression.

6.2 Future Enhancements

Several enhancements and development of the project would improve the functionality of the project.

6.2.1 Multi digit Operand Support

The converter would be more useful if it could handle multi-digit numbers character variable names. To properly detect and handle operands longer than a single character, this improvement would necessitate changes to the tokenization logic in the conversion methods.

6.2.2 Additional Conversion

The conversion from prefix to postfix and vice versa along with its real time algorithm visualization would enhance the features of the project.

6.2.3 Enhanced Visualization

The current implementation of the visualization displays simple algorithm traces in a table. In future, the push and pop operations could be displayed on a graphical stack representation in real time. This would enhance the learning experience of the users.

6.2.4 History Persistence

The implementation of file-based storage for conversion history would allow users to reload their past conversions. This would be useful for students to review their past conversions.

6.2.5 Mobile Application

A significantly larger audience may use the Expression Notation Converter if a mobile version for iOS or Android was created using Qt for Mobile. A web-based version, on the other hand, would do away with the requirement for installation completely, enabling students to utilize the converter straight from a browser on any device.

References

(n.d.). Retrieved from Tutorialspoint: https://www.tutorialspoint.com/data_structures_algorithms/expression_parsing.htm

(n.d.). Retrieved from CalConT: https://www.calcont.in/

(n.d.). Retrieved from Expression Tree Generator: https://expression-generator.netlify.app/

(n.d.). Retrieved from Free Online Calculator: https://www.free-online-calculator-use.com/

(n.d.). Retrieved from Scanftree: https://scanftree.com/Data_Structure/prefix-postfix-infix-online-converter

Wolf, C. E. (n.d.). Retrieved from Mathcenter Oxford: https://mathcenter.oxford.emory.edu/site/cs171/shuntingYardAlgorithm/

Appendix

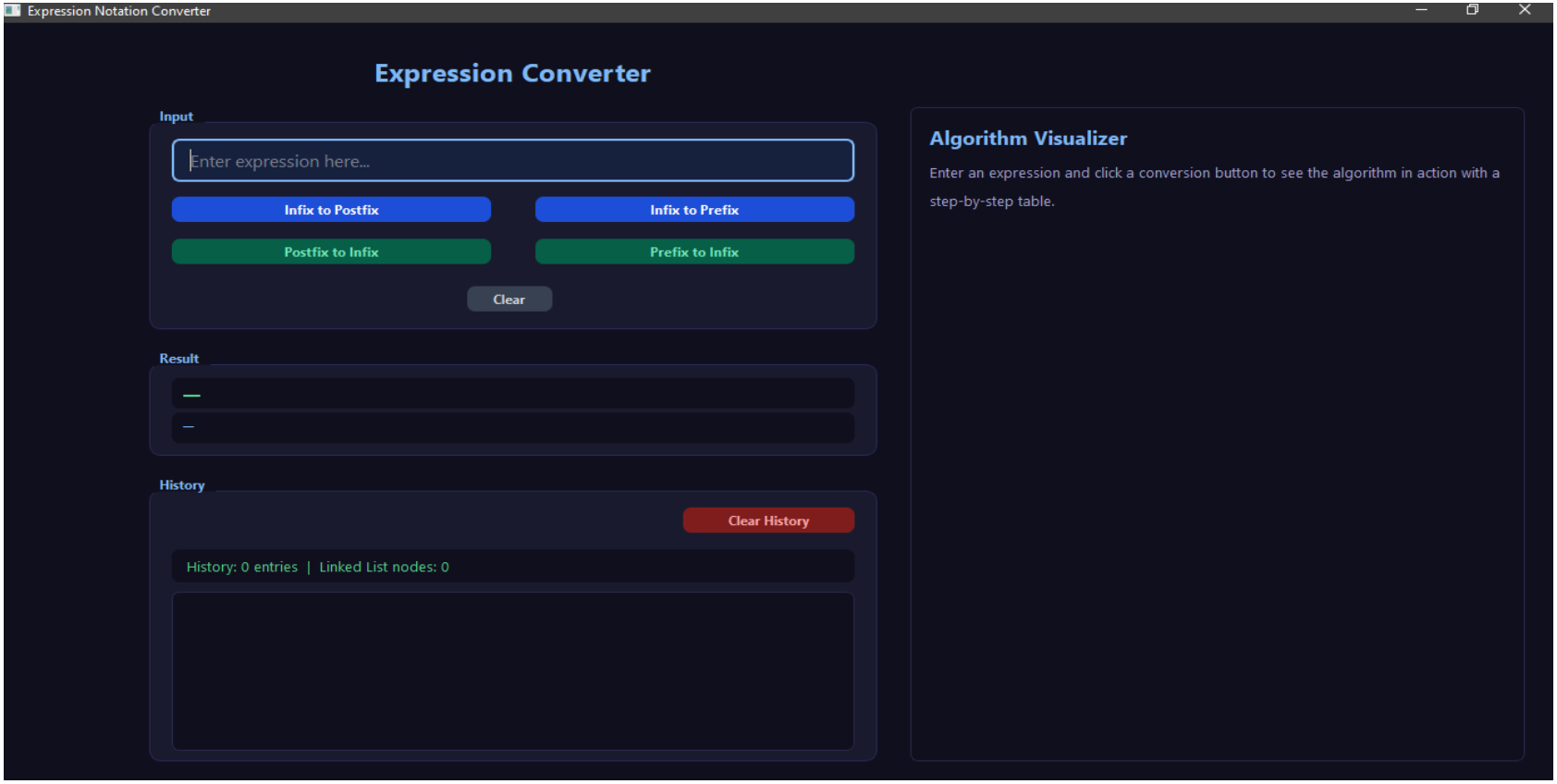

Figure 2 Main Window

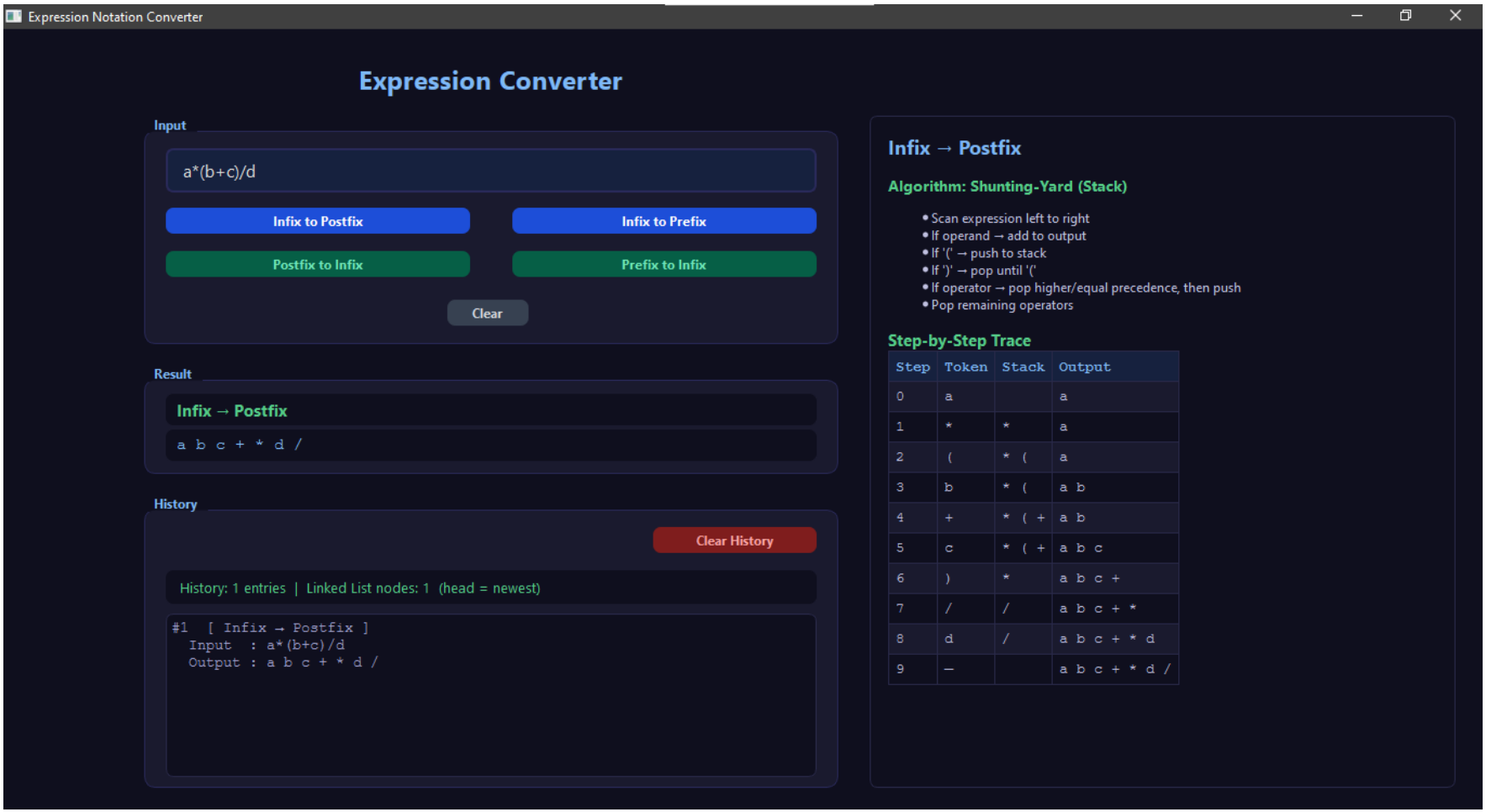

Figure 3 Infix to Postfix

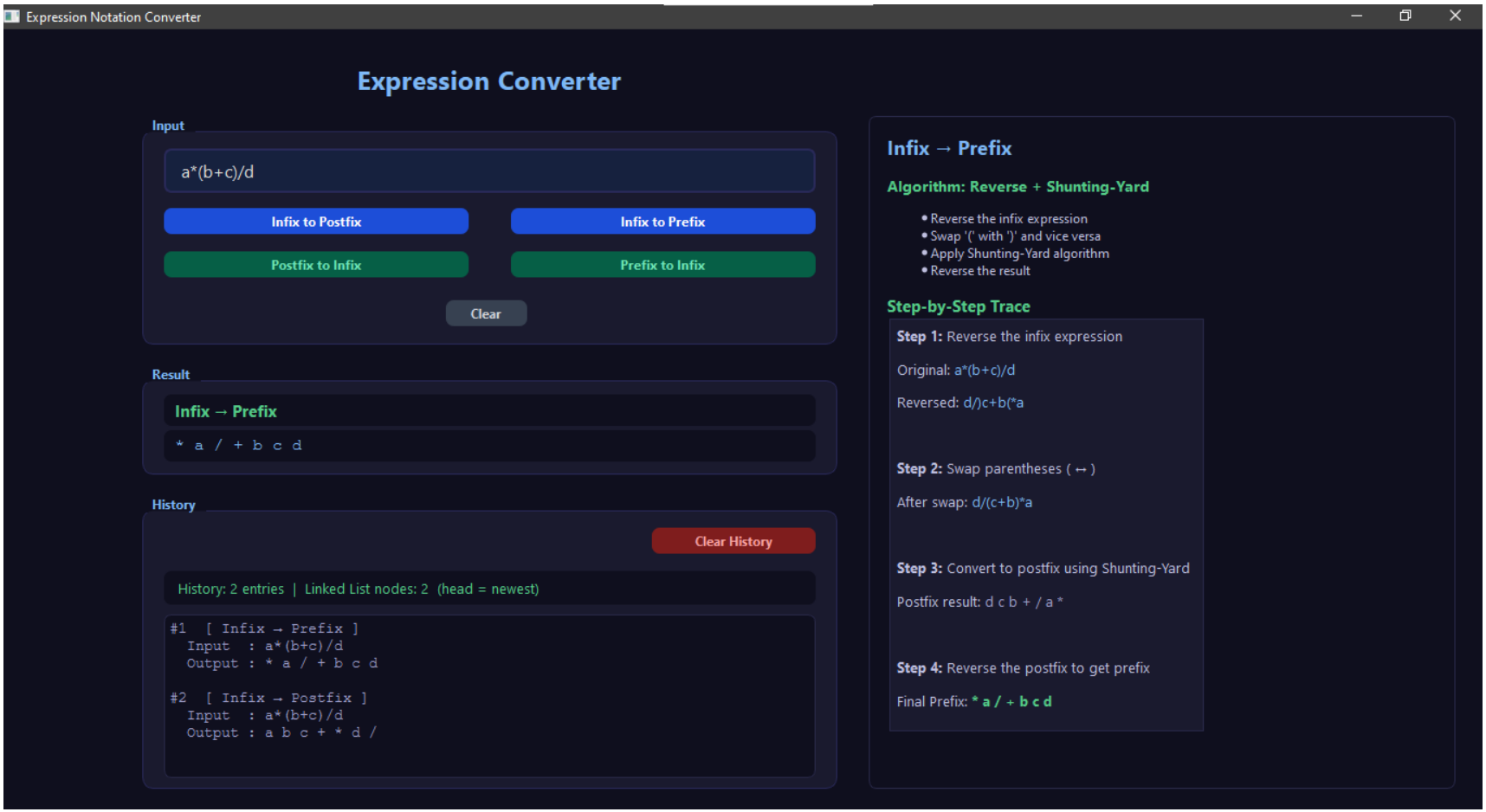

Figure 4 Infix to Prefix

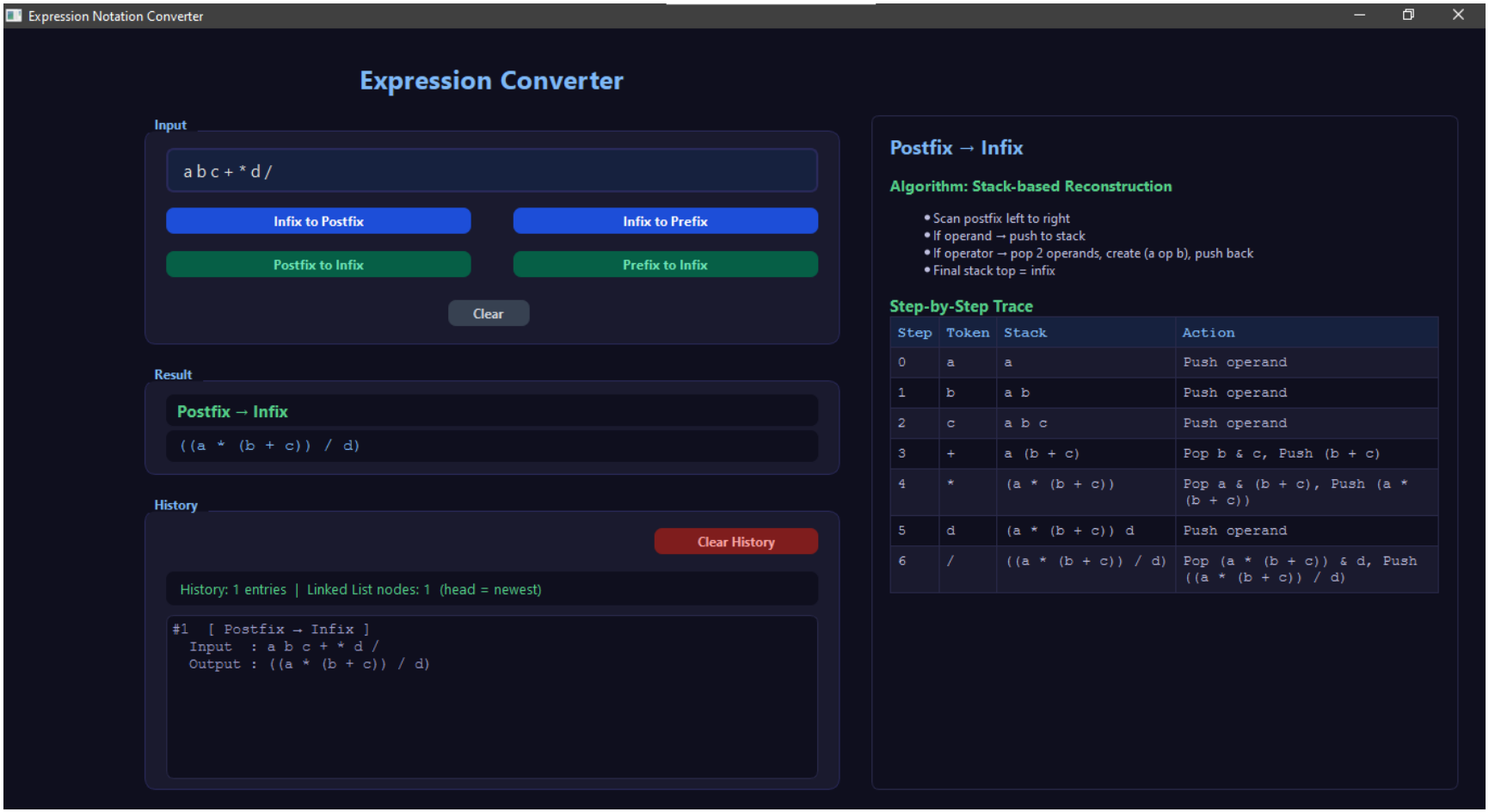

Figure 5 Postfix to Infix

Figure 6 Prefix to Infix